Laboratório #01

Introdução ao Android

Objectivos

- Introduzir o desenvolvimento de aplicação Android
- Familiazar o estudante com as ferramentas de desenvolvimento Android: Android Studio e Gradle.

1. Introdução

1.1. O que é o Android

O Android oferece um conjunto completo de software para dispositivos móveis: um sistema operativo, *middleware* e as principais aplicações móveis. Leia mais: http://android.com/about/

1.2. Visão geral do Android

O Android é uma pilha de software de código aberto baseada em Linux criada para uma ampla variedade de dispositivos e factores de forma. O diagrama a seguir mostra os principais componentes da plataforma Android.

joao.costa@isptec.co.ao

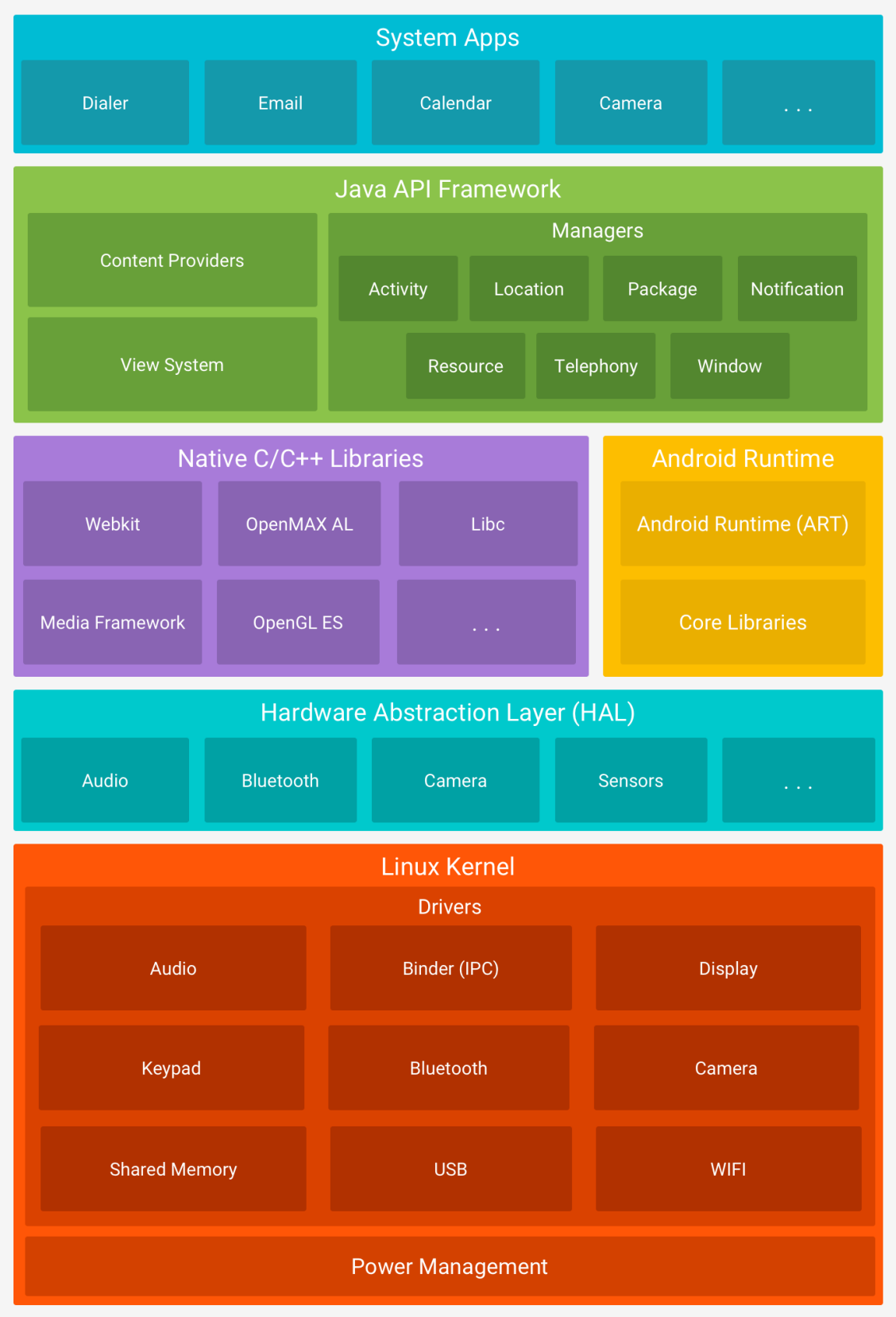


Figure 1: Pilha de software do Android

Leia mais em: https://developer.android.com/guide/platform#art

Recursos notáveis

- Framework de aplicação orientada a objetos baseada em Java
  - Aplicações sobre bibliotecas principais Java em execução em uma máquina virtual *Android Runtime (ART)*
- Implementação Java altamente optimizada
  - Muito eficiente em termos de memória e desempenho
  - Altamente ajustado às limitações de hardware pequeno
- Baseado em uma versão modificada do kernel Linux
- Ambiente de desenvolvimento rico
  - Emulador de dispositivo, ferramentas para depuração, criação de perfil, integração IDE avançada

Segurança e permissões

- Cada aplicação instalada com ID único de utilizador e grupo
  - Cada ficheiro de aplicação é privado
  - A partilha deve ser feita explicitamente
- Aplicações em sandbox em VMs separadas
- Princípio do menor privilégio
  - As aplicações devem declarar as permissões de que precisam
  - O sistema solicita o consentimento do utilizador no momento da instalação

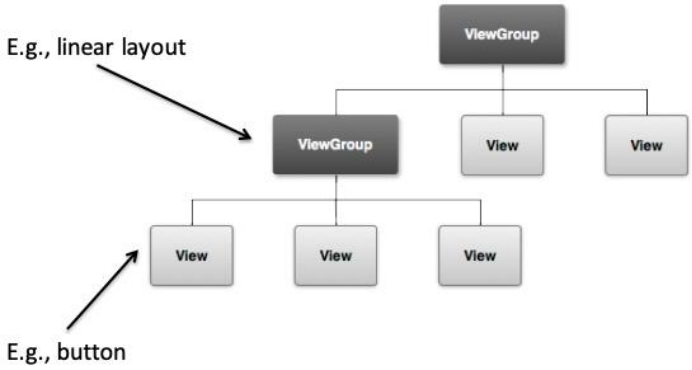
1.3. Anatomia da aplicação Android

Exemplo de componente

- Actividades são como páginas num website
  - Fornece uma interface para o utilizador interagir com a aplicação e realiza uma acção
  - 

Views

- Views: elementos de interface do utilizador, organizados hierarquicamente.
  - Dois tipos: **layouts** e **widgets**



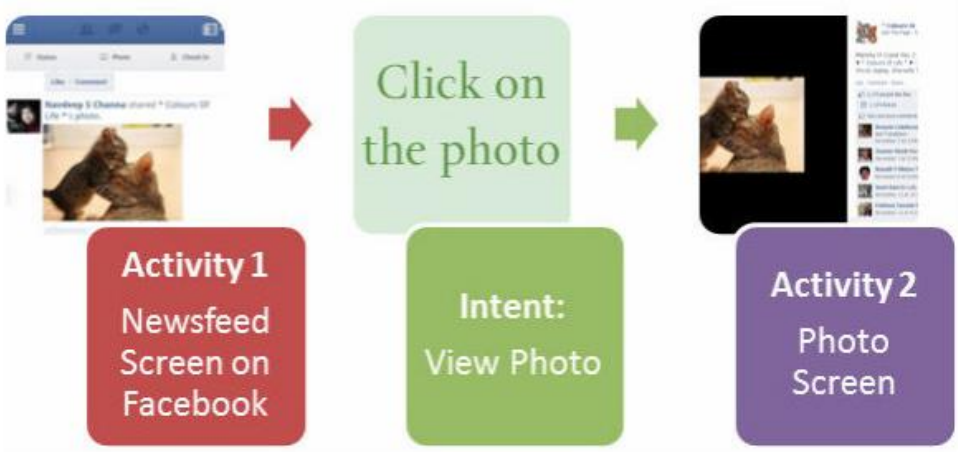
Componentes

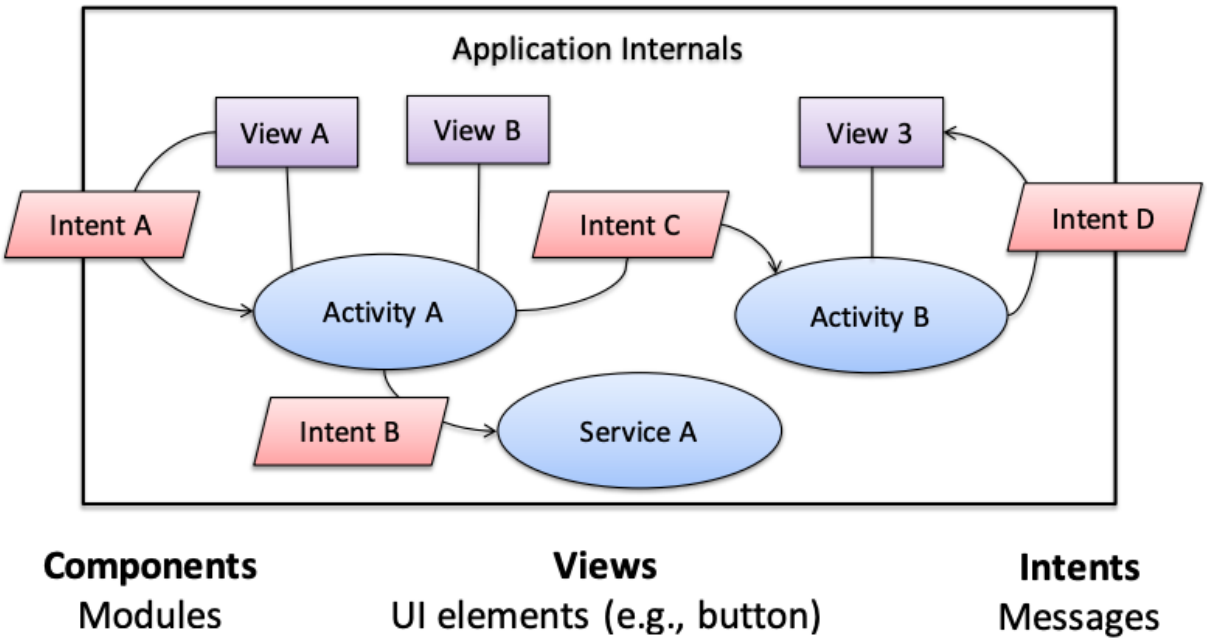
- Existem quatro tipos diferentes de componentes de aplicação
  - Cada tipo tem um propósito diferente e um ciclo de vida distinto



Intents (intenções)

- Mensagens que permitem a comunicação entre componentes



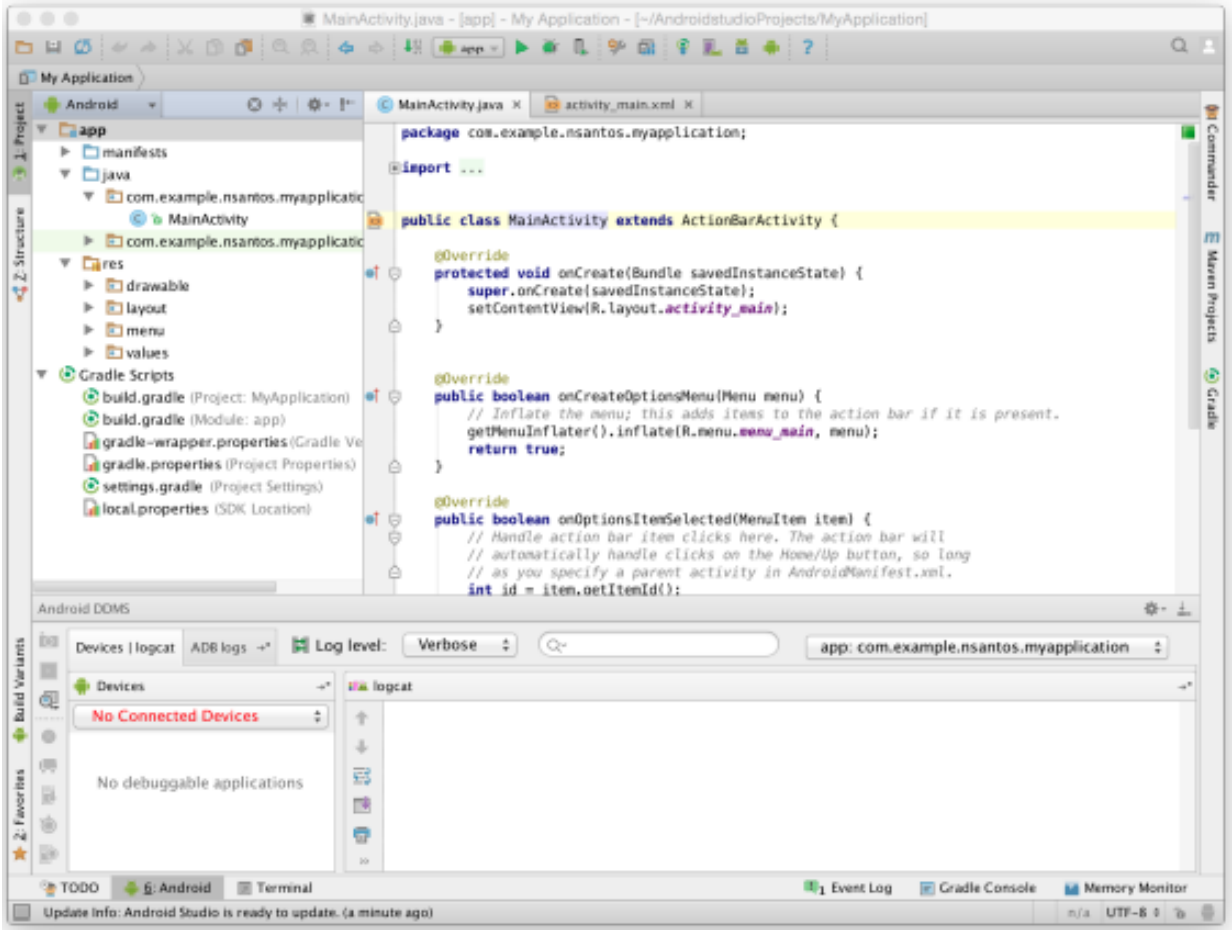


1.4. Desenvolvimento da aplicação

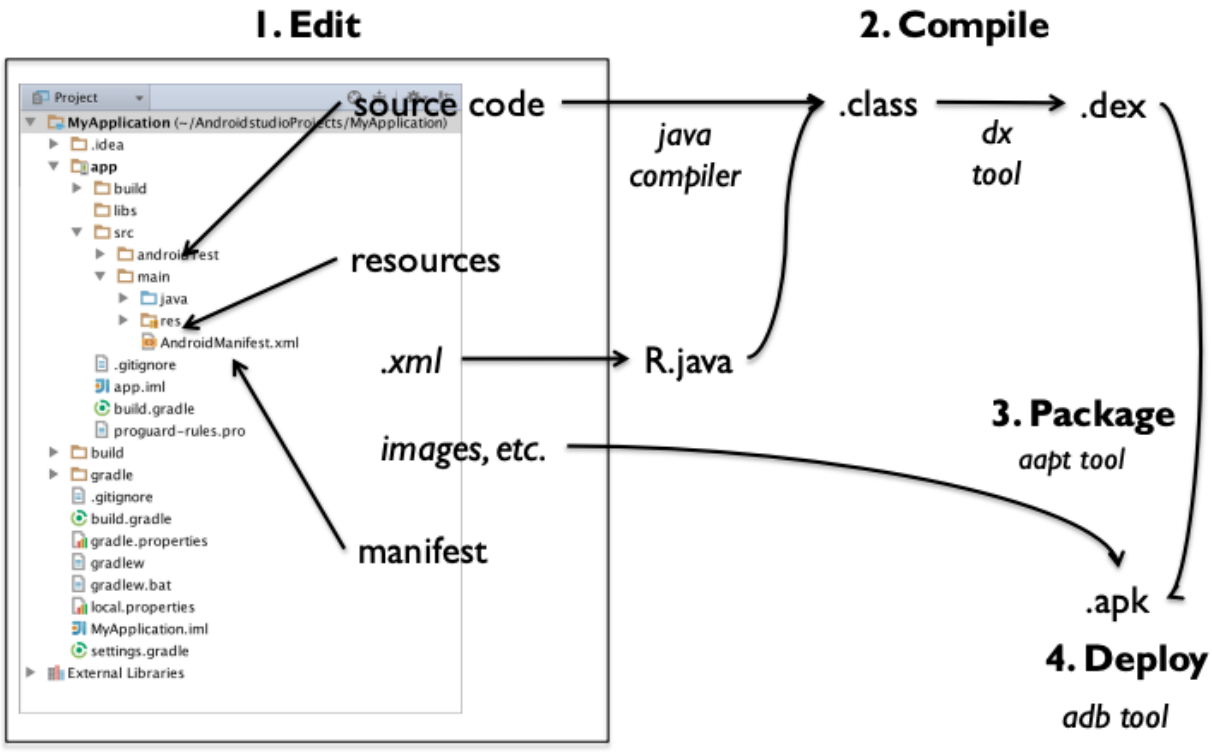
Ferramentas de desenvolvimento

- Android Software Development Kit (SDK)
  - Ferramentas para criar, compilar e empacotar aplicações
  - Emulador de dispositivo
  - Ferramentas para criar dispositivos virtuais Android (AVDs , Android Virtual Devices)
  - Ferramenta Android Debug Bridge (ADB)
- Android Studio
  - IDE completo baseado em IntelliJ
  - Criar, compilar, depurar e implantar aplicações Android
  - Criar e iniciar AVDs
  - Edição especializada de ficheiros de recursos

Android Studio



Processo de desenvolvimento



Manifesto

- `AndroidManifest.xml` define o esqueleto de uma aplicação

```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.myfirstapp"
    android:versionCode="1"
    android:versionName="1.0" >
    <application
        android:allowBackup="true"
        android:icon="@drawable/ic_launcher"
        android:label="@string/app_name"
        android:theme="@style/AppTheme" >
        <activity
            android:name="com.example.myfirstapp.MainActivity"
            android:label="@string/app_name" >
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

Recursos

- Inclue imagens e determinados ficheiros de configuração XML

– e.g., *res/layout/activity_main.xml*

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="horizontal" >

    <EditText android:id="@+id/edit_message"
            android:layout_height="wrap_content"
            android:layout_width="0dp"
            android:layout_weight="1"
            android:hint="@string/edit_message" />

    <Button
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="@string/button_send"
            android:onClick="sendMessage" />

</LinearLayout>
```

- Uma referência a um novo recurso é criada automaticamente em R.java

Código-fonte

- O código-fonte implementa os componentes da aplicação

– E.g., *src/com/example/myfirstapp/MainActivity.java*

```
package com.example.myfirstapp;

import android.app.Activity;

public class MainActivity extends Activity {

    public final static String EXTRA_MESSAGE = "com.example.myfirstapp.MESSAGE";

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }

    @Override
    public boolean onCreateOptionsMenu(Menu menu) {
        // Inflate the menu; this adds items to the action bar if it is present.
        getMenuInflater().inflate(R.menu.main, menu);
        return true;
    }
}
```

Links Úteis

- Desenvolvedores Android: http://developer.android.com
  - Treinamento: http://developer.android.com/training/index.html
  - Guias de API: http://developer.android.com/guide/components/index.html
  - Referência: http://developer.android.com/reference/packages.html
- Lembre-se, o Google é seu amigo

2. Exercício I – Hello World

Implemente uma aplicação Hello World que renderize as duas actividades mostradas na Figura 2. A actividade do lado esquerdo é mostrada quando a aplicação é iniciada e apresenta uma caixa de texto e um botão "Enviar". Quando o utilizador digita uma string na caixa de texto e pressiona o botão, a actividade do lado direito é criada, ecoando a string de entrada de volta para o utilizador.

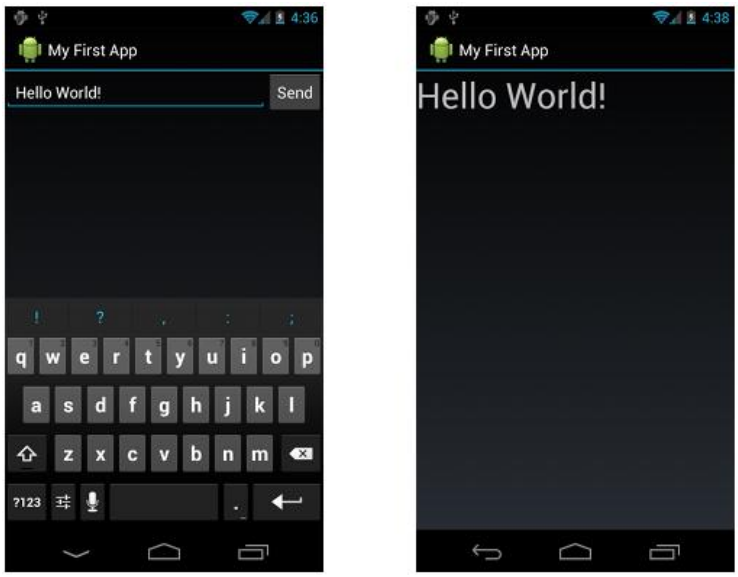


Figure 2: Actividades implementadas pela aplicação Hello World

2.1. Criar um projecto Android

O primeiro passo para implementar a aplicação é criar um projecto Android. Utilizaremos a versão mais recente do Android Studio:

- Abra o Android Studio e selecione "Start a new Android Studio project". No painel "Choose your project", selecione "Empty Activity".
- Em "Configure your project", atribua nomes à sua aplicação (Hello World) e o nome do pacote (ao.co.isptec.aplm.exercise_1). Deixe todos os outros campos inalterados.

Navegue até a visualização "Project" (clique no botão suspenso superior esquerdo chamado "Android") e explore a árvore de directórios do projecto. Descreva a função dos seguintes ficheiros (Dica: leia "Build your first Android app", https://developer.android.com/training/basics/firstapp):

- app/src/main/res/layout/activity_main.xml
- app/src/main/java/ ao.co.isptec.aplm.exercise_1/MainActivity.java
- app/src/res/AndroidManifest.xml
- app/build.gradle

2.2. Executar a aplicação

Para testar sua aplicação, pode utilizar o emulador ou um dispositivo real.

Emulador: Para executar a aplicação no emulador, execute as seguintes etapas:

- Crie um dispositivo virtual Android (AVD): (a) inicie o gestor de dispositivo virtual do Android em "Tools > AVD Manager", (b) clique em "Create Virtual Device", (c) na janela "Select

hardware", escolha uma configuração de dispositivo (por exemplo, Nexus 5X), (d) primeiro clique em "x86 Images" (isso permitirá que adapte o tamanho da RAM do AVD), selecione uma imagem do sistema para AVD (por exemplo, Pie, API 28 - ABI x86) , clique em próximo, (e) para configurar o espaço de memória alocado para o AVD (por exemplo, para 512 MB), clique em "Show Advanced Settings" e execute essa alteração de acordo e (f) conclua.

- Inicie o AVD recém-criado: no painel da gestão de dispositivo virtual, selecione o AVD e pressione o ícone "play" (pode levar algum tempo para o emulador inicializar).
- Execute a aplicação no Android Studio: (a) no Android Studio, clique em "Run" , (b) na janela "Select Deployment Target", selecione o AVD em execução na lista " Connected Devices " e clique em ok. A aplicação deve aparecer no emulador.

Dispositivo Real: Se possui um dispositivo real e um cabo USB, faça o seguinte:

- *Configure seu dispositivo*: (a) conecte seu dispositivo à sua máquina de desenvolvimento com um cabo USB, (b) ative a "USB debugging" em seu dispositivo – na maioria dos dispositivos com Android 3.2 ou anterior, esta opção está localizada em "Configurações > Aplicações > Desenvolvimento; no Android 4.0 e mais recente, está em "Configurações > Opções do desenvolvedor". Observação: no Android 4.2+, as "Opções do desenvolvedor" ficam ocultas por padrão. Para disponibilizá-lo, proceda conforme descrito em https://developer.android.com/training/basics/firstapp .
- *Execute a aplicação no Android Studio*: (a) no Android Studio, clique em "Run", (b) na janela "Select Deployment Target", escolha seu dispositivo na lista "Connected Devices".

É possível gerir vários AVDs no Android Studio ou em uma interface de linha de comando. Para saber mais sobre este tópico, leia em https://developer.android.com/studio/run/managing-avds .

2.3. Debug da aplicação

Adicione a seguinte linha de código ao final do método onCreate da classe MainActivity. Também pode precisar adicionar "`import android.util.Log;`" para a lista de importação de pacotes no início do ficheiro. Em seguida, reimplante o código da aplicação clicando no botão "Run" (pode manter o AVD em execução). Confira a mensagem apresentada no painel do logcat (selecione a guia "Logcat" na parte inferior do IDE e filtre as mensagens de saída com base no nome do pacote da aplicação, ou seja, "ao.co.isptec.aplm.exercise_1").

```
Log.d("MainActivity", "Minha primeira mensagem de depuração.");
```

Coloque um ponto de interrupção em qualquer lugar no código da aplicação (por exemplo, na linha Log.d) e execute o programa na visualização do depurador. Explore as funcionalidades do depurador inspecionando o estado da aplicação e retomando a execução do processo. Enquanto a aplicação está parada, é possível inspecionar as threads em execução no emulador, a utilização de recursos, etc.

2.4. Construção de uma interface do utilizador simples

Implemente a actividade mostrada no lado esquerdo da Figura 2. Para isso, deve modificar o layout padrão da interface do utilizador (UI) para incluir um campo de texto e um botão. Isso será feito em XML (`res/layout/activity_main.xml`). É possível especificar o layout de uma actividade utilizando a GUI do Android Studio, mas deixamos isso como exercício.

No Android, conforme mostrado na Figura 3, o layout da UI de uma actividade consiste em uma árvore hierárquica de elementos, cada um dos quais é chamado de **View**. As folhas da árvore correspondem a elementos únicos da interface do utilizador (por exemplo, botões). Subindo na árvore, esses elementos são agregados em grupos (**ViewGroup**), que por sua vez podem ser organizados em grupos de nível superior. A raiz da árvore representa o painel de nível superior da actividade. Uma ViewGroup também é uma View.

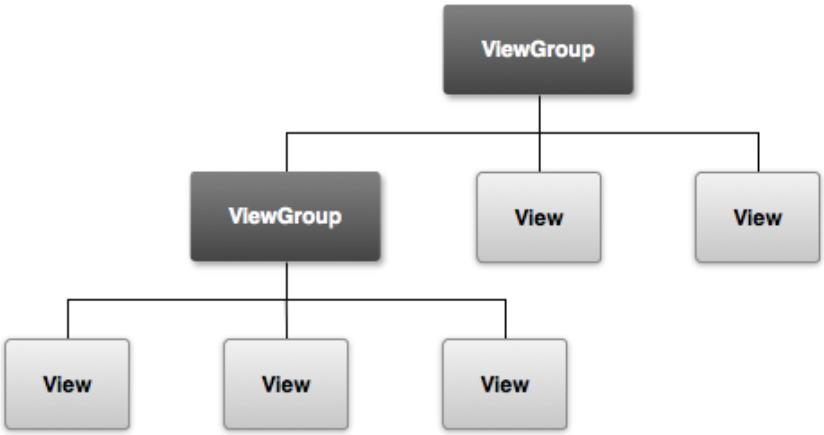


Figure 3: Representação genérica de uma árvore de layout da UI

Diferentes elementos **ViewGroup** especificam diferentes arranjos para suas visualizações filhas. Por exemplo, a ViewGroup chamado **LinearLayout** apresenta view filhas em sequência, de acordo com a maneira como elas aparecem no XML. Essa sequência pode ter orientação vertical ou horizontal, conforme especificado pelo atributo `android:orientation`. Para obter mais informações, consulte o link https://developer.android.com/develop/ui/views/layout/declaring-layout .

Implemente o novo layout da actividade principal do Hello World:

- Esboce a árvore da UI: desenhe uma árvore da UI que reflita o layout da actividade mostrada no lado esquerdo da Figura 2. Dica: utilize as views **Button**, **EditText** e **LinearLayout**.
- Escreva o código XML correspondente: Dica: use o conteúdo do arquivo activity_main.xml (em anexo) e veja como ele reflete sua árvore de UI.
- Execute novamente a aplicação no Android Studio.

2.5. Iniciar outra actividade

Conclua sua aplicação iniciando outra actividade quando o utilizador pressionar o botão "Enviar":

- Edite `MainActivity` e defina-o para o código no correspondente ficheiro disponibilizado.
- Crie uma nova actividade (File > New > Activity > Empty Activity) chamada DisplayMessageActivity. Utilize o código no correspondente ficheiro disponibilizado para o layout da actividade e este código para a implementação da actividade.
- Execute novamente a aplicação a partir do Android Studio: sua aplicação deve agora produzir a saída apresentada na Figura 2. Parabéns!

Responda as seguintes questões:

1. O quê que alterou no ficheiro `AndroidManifest.xml`?
2. Como o evento clique é tratado?
3. Como identificar e obter ou definir componentes de interface do código Java?
4. Qual é a finalidade da `intent` da variável criada na classe `DisplayMessageActivity`?

3. Exercício II – Desmistificando Gradle

Para gerir o processo de construção de um projecto, o Android utiliza o Gradle (https://gradle.org/), um sistema de automação de construção de código aberto semelhante ao Ant (https://ant.apache.org/). É importante entender como o Gradle funciona porque o ajudará a corrigir problemas relacionados à compilação mais rapidamente e permitirá que gere seus projectos a partir da linha de comando.

3.1. Comandos básicos do Gradle

Primeiro, baixe-o e instale-o em um directório local. Observe que, primeiro, utilizará a distribuição padrão do Gradle e, posteriormente, a distribuição que acompanha o Android Studio.

- Baixe a distribuição gradle-5.2.1 (https://services.gradle.org/distributions/gradle-5.2.1-all.zip) e descompacte-a em algum directório local (por exemplo, `~/bin`). Assumimos que o Java SDK está configurado correctamente em sua máquina.
- Crie uma variável de ambiente chamada `GRADLE_HOME` e aponte-a para a pasta de instalação do `gradle`; adicione `$GRADLE_HOME/bin` à variável de ambiente `PATH`. Pode então executar o comando `gradle` de qualquer diretório.

Em uma janela de terminal ou linha de comandos, digite os seguintes comandos e saiba o que eles fazem:

```
gradle -v
gradle -q help
gradle -q tasks
gradle -q properties
```

Crie uma pasta para um projecto hipotético (por exemplo, `exemplo1`) e crie um ficheiro chamado `build.gradle`. Este ficheiro pode ser visto como Makefile do Gradle (lembra as práticas de Sistemas Operativos????). Adicione o seguinte conteúdo ao ficheiro:

```
task compileTask {
    doLast {
        System.out.println "compiling..."
    }
}
```

Repita o comando `gradle -q tasks --all` e localize a tarefa definida em `build.gradle`. Em seguida, execute a tarefa:

```
gradle -q compileTask
```

Modifique `build.gradle` da seguinte forma:

```
defaultTasks 'buildTask'

task compileTask {
    doLast {
        System.out.println "compiling..."
    }
}

task buildTask (dependsOn:compileTask) {
    doLast {
        System.out.println "building..."
    }
}
```

Interprete a saída do Gradle ao emitir os seguintes comandos:

```
gradle -q compileTask
gradle -q buildTask
gradle -q
```

Como já deve ter adivinhado, as tarefas são os blocos de construção para gerir um projecto. Fazem coisas como: compilar, construir, executar testes, empacotar, etc. No entanto, não precisa escrever todas essas tarefas manualmente. Na verdade, o Gradle pode ser estendido com plugins que definem tarefas para tipos de projetos específicos: plugin Java, plugin WAR, plugin Android, etc. Em seguida, construirá um projecto Java usando o plugin Gradle Java.

3.2. Simples projecto Java com Gradle Java plugin

Para introduzir o Gradle Java plugin, implementaremos uma biblioteca Java simples. Esta biblioteca implementa uma classe de domínio chamada `Cota` para gerir citações famosas. Uma cotação contém um ID, o autor da cotação e o texto da cotação.

Crie um directório para o projecto chamado `exemplo2`. Coloque o ficheiro `build.gradle` no directório do projecto e o ficheiro `Quota.java` em um subdiretório do projecto `src/main/java/ao/co/isptec/aplm/quota`.

Responda as seguintes perguntas:

- Qual é o propósito da declaração `apply plugin: "java"` em `build.gradle`?
- Execute `gradle tasks --all` e explore as tarefas definidas pelo plugin Gradle Java. Em particular, execute `gradle build` e explique o resultado desta tarefa.
- Por que o `Quota.java` é colocado nesse directório específico?
- Explique o significado das dependências e repositórios de blocos no ficheiro `build.gradle`.

Modifique a classe para implementar o seguinte método Main:

```
public static void main(String[] args) {
```

```
        System.out.println("Aplicação Quota em execução");
        Quota q = new Quota();
        q.setId(new Long(1));
        q.setAutor("Joao Costa");
        q.setTexto("Deus é vida.");
        System.out.println(q);
}
```

Para executar este código do Gradle, precisamos adicionar um plugin adicional. Modifique `build.gradle` adicionando o código a seguir e execute "`gradle run`". Analise o que aconteceu. Dica: leia " https://docs.gradle.org/current/userguide/application_plugin.html ".

```
apply plugin:'application'
mainClassName = "ao.co.isptec.aplm.quota.Quota"
```

3.3. Múltiplo projectos Java

Normalmente, um projecto não é um software monolítico, mas consiste em vários componentes, por exemplo, bibliotecas ou APIs. Gradle pode lidar facilmente com projectos complexos, estruturando-os em subprojectos. Cada subprojecto é colocado em um subdirectório e gerido por um ficheiro `build.gradle` específico.

Para entender como o Gradle gere subprojectos, baixe e descompacte o ficheiro de projecto `exemplo3.zip` e responda as seguintes perguntas:

- Em quais subprojectos este projecto está dividido? Determine como executar o Gradle para construir (i) todo o projecto e (ii) cada subprojecto individualmente.
- Identifique as dependências de cada subprojecto e desenhe o gráfico de dependência do projeto.

3.4. Gradle e Android Studio

O Android Studio aproveita o Gradle para gerir seus projectos utilizando o plug-in do Gradle para Android. Consulte o projecto Hello World implementado no Exercício I. Com a ajuda do tutorial "Gradle Tutorial: https://rominirani.com/gradle-tutorial-part-6-android-studio-gradle-c828c5639bb ", execute os seguintes exercícios:

- Identifique dentro da estrutura interna do projecto, quais ficheiros são relevantes para o Gradle e qual é a sua função.
- Limpe e reconstrua o projecto usando o painel de tarefas do Gradle do Android Studio.
- Repita as operações anteriores usando o console Gradle.
- Execute novamente as tarefas de limpeza e compilação usando um terminal externo e a distribuição Gradle empacotada no Android Studio (Dica: explore o script `gradlew`).

Neste ponto, deve ficar claro que não há mágica sob o capô. Para referência futura, considere os seguintes guias:

- https://developer.android.com/sdk/installing/studio-build.html
- https://gradle.org/documentation
- http://gradle.org/docs/current/userguide/java_plugin.html
- http://tools.android.com/tech-docs/new-build-system/user-guide

4. Exercício III – Aplicação de lista de tarefas

Implemente uma aplicação simples de lista de tarefas que consiste em uma única actividade. A actividade deve mostrar duas visões organizadas verticalmente: uma visão `EditText` e uma visão `ListView`. O utilizador digita uma string na visão `EditText` e o `ListView` será actualizado com a nova string colocada no topo da lista.

- **Dica 1:** Use um `ArrayAdapter` para vincular um `ArrayList` que contém a lista de strings ao `ListView`.
- **Dica 2:** Para capturar a acção de entrada do utilizador na visão `EditText`, implemente `setOnKeyListener` desta visão.